Prerequisites and objectives

- Prerequisites are arrays, heaps, recursion, expectation, and Chapter 1 complexity definitions.
- Explain each algorithm's invariant, correctness, and cost. Distinguish stability, in-place storage, and the comparison model.
- Choose using existing order, key domain, memory limits, and worst-case requirements.

Sorting contract and stability

- The output must be a permutation of the input and satisfy `A[i]≤A[i+1]`. Order alone does not rule out lost or duplicated elements.
- Stability preserves the original relative order of equal-key records. Sorting `(2,A),(1,X),(2,B)` must leave A before B.
- Here in-place means O(1) extra data-array space. Recursion-stack space is reported separately.
- The comparator must express a consistent order. C++ sorting requires strict weak ordering. Using $\leq$ as an ordinary comparator violates that requirement.

Insertion-sort invariant

1. Before iteration i, `A[0,i)` is sorted and is a permutation of the original first i elements.
2. Save key=A[i]. Shift prefix elements strictly greater than key rightward until its insertion slot is available.
3. Write key there, obtaining sorted prefix `A[0,i+1)`.

Moving only strictly greater keys preserves stability. Example `[5,2,4,2]` becomes `[2,5,4,2]`, `[2,4,5,2]`, then `[2,2,4,5]`.

Insertion sort and inversions

- An inversion is a pair i<j with A[i]>A[j]. Each rightward shift removes one inversion involving the current key.
- With I inversions, standard insertion sort takes $\Theta(n+I)$.
- Sorted input has I=0 and best time $\Theta(n)$. Distinct reverse-sorted input has $I=n(n-1)/2$ and worst time $\Theta(n^2)$.
- The inversion count explains suitability for small or nearly sorted inputs without an unsupported claim about average speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Selection sort: fixed comparisons, few swaps

At round i find a minimum in A[i,n) and swap it with A[i]. The invariant is that A[0,i) is sorted and every key there is no larger than every remaining key. Selecting the minimum extends the invariant by one; swaps preserve the input permutation.

Regardless of input order, there are $(n-1)+(n-2)+\dots+1=n(n-1)/2$ comparisons. The iterative version performs at most n−1 swaps and uses O(1) auxiliary space, although moving a large record can still be costly.

The usual swap version is unstable: [2A,2B,1] becomes [1,2B,2A] after the first swap. Equal-key records reverse. A stable variant needs different movement and cannot inherit the same swap-cost claim unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bubble sort and inversions

Compare adjacent records left to right, swapping only when the left key is strictly larger. At each pass the maximum remaining key reaches the right end. Starting with [3,1,2,0], three passes produce [1,2,0,3], [1,0,2,3], then [0,1,2,3].

Each adjacent strict-inversion swap removes exactly one inversion, so the total swap count equals the initial inversion count I=5. Strict comparison preserves the order of equal keys. An unchanged pass permits early exit: best $\Theta(n)$, worst $\Theta(n^2)$, $O(1)$ auxiliary space for the iterative version. Without early exit, even sorted input can require $\Theta(n^2)$ comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shell sort gap trace



Insertion-sort each residue class. A final gap of 1 ensures global order.

Unstable: gap=2 changes [5A,5B,2,8] to [2,5B,5A,8].

Equal keys A and B cross between groups; the final stable pass cannot restore them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shell sort invariants and cost assumptions

```
shell_sort(A,gaps):
    require gaps are positive integers and last gap is 1
    for gap in gaps:
        for i=gap..n-1:
            x=A[i]; j=i
            while j>=gap and A[j-gap].key>x.key:
                A[j]=A[j-gap]; j-=gap
            A[j]=x
```

Each pass insertion-sorts indices in each residue class modulo gap. At the final gap 1 all records belong to one class, so the whole array is sorted; shifts and reinsertion preserve the records.

Costs depend on the selected gap sequence; there is no single tight complexity for every sequence. With gaps=[1], this is insertion sort with $\Theta(n^2)$ worst case. Movement across different classes can reverse equal keys, so ordinary Shell sort is unstable even if each individual insertion pass is stable.

Quicksort partition contract

- Three-way partition divides the range around pivot value p into `<p`, `=p`, and `>p`. Recurse only on the outer regions.
- Maintain `[l,lt)<p`, `[lt,i)=p`, unclassified `[i,gt)`, and `[gt,r)>p`.
- For a smaller value, swap to lt and increment lt and i. For equality, increment i. For a larger value, decrement gt, swap there, and keep i to inspect the incoming value.
- The measure gt−i decreases each iteration. At termination every element is classified. Partition time is $\Theta(r-l)$.

Three-way partition indices and recursion contract

```
partition(A,l,r,p):
    lt=l; i=l; gt=r
    while i<gt:
        if A[i]<p: swap(A[lt],A[i]); lt+=1; i+=1
        else if A[i]>p: gt-=1; swap(A[i],A[gt])
        else: i+=1
    return (lt,gt)
quicksort(A,l,r):
    if r-l<2: return
    choose p from A[l:r]
    (lt,gt)=partition(A,l,r,p)
    quicksort(A,l,lt); quicksort(A,gt,r)
```

For [4,2,4,7,1,4], states (A, lt, i, gt) are initially (A,0,0,6), then (A,0,1,6), ([2,4,4,7,1,4],1,2,6), (same,1,3,6), ([2,4,4,4,1,7],1,3,5), (same,1,4,5), and ([2,1,4,4,4,7],2,5,5). Recurse only on [0,2) and [5,6).

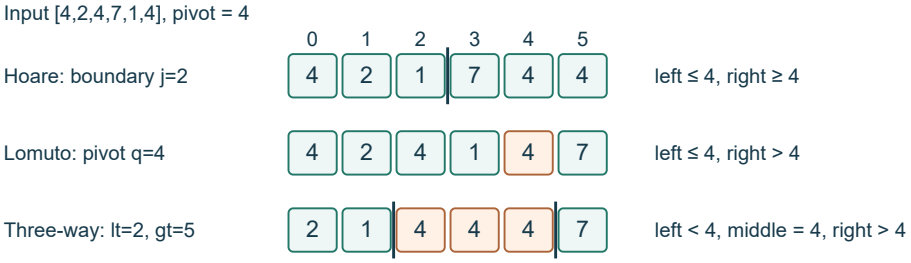
Three-way partition indices and recursion contract (continued)

Choosing the pivot from the subarray guarantees a nonempty equal region and smaller recursive problems. An absent extreme pivot could leave a recursive side equal to the original range. Expected bounds require random selection, not a fixed middle-position value. Smaller-side recursion bounds stack depth but leaves quadratic worst-case time possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three partition schemes, three contracts

One input, three partition contracts



Hoare returns a boundary j, not necessarily the pivot's final position.

Lomuto and three-way partition use different recursive ranges, especially with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Do not interchange Hoare and Lomuto boundaries

Use half-open [l,r). A traditional Hoare bidirectional scan can return j with A[l,j+1)≤p and A[j+1,r)≥p. j is a boundary; A[j] need not be the pivot object or its final position. Recurse on [l,j+1) and [j+1,r). Another bidirectional variant swaps the first pivot into its final position; when it returns that pivot position, recursion must exclude it rather than use this boundary contract.

Lomuto chooses the last key, moves keys ≤p left, and finally puts that pivot at q. Then [l,q)≤p, A[q]=p, and [q+1,r)>p; recursion excludes q. The three-way scheme returns the entire equality interval [lt,gt), which both recursive calls skip.

On all-equal keys this Lomuto version splits into sizes n−1 and 0 and takes $\Theta(n^2)$. The next Hoare version stops on equality on both sides and splits near the middle. Changing equality tests can alter balance or termination, so these claims are version-specific.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hoare bidirectional scan: executable contract

```
hoare(A,l,r):              // require r-l>=2
    p=A[l]; i=l-1; j=r
    while true:
        do i+=1 while A[i]<p
        do j-=1 while A[j]>p
        if i>=j: return j
        swap(A[i],A[j])
quicksort_hoare(A,l,r):
    if r-l<2: return
    j=hoare(A,l,r)
    quicksort_hoare(A,l,j+1)
    quicksort_hoare(A,j+1,r)
```

On [4,2,4,7,1,4], first i=0,j=5 swap equal 4s; then i=2,j=4 swap 4 and 1, giving [4,2,1,7,4,4]. Next i=3,j=2, so return j=2. Left [4,2,1]≤4 and right [7,4,4]≥4. Both recursive ranges are nonempty and shorter.

Each exchange puts the left-discovered ≥p key and right-discovered ≤p key on their appropriate sides. The indices move inward and stop when crossed. Equality also stops scans, and indices advance after a swap; omitting that progress can loop forever on equal keys.

Quicksort worst-case and expected time

- For `[4,2,4,7,1,4]` and p=4, a partition is `[2,1] | [4,4,4] | [7]`. Order inside regions is unspecified.
- With distinct keys and extreme pivots, `T(n)=T(n−1)+Θ(n)`, yielding worst-case $\Theta(n^2)$.
- For a fixed distinct-key input, uniformly random pivots in every subproblem give expected Θ(n log n). In the classic model comparing the pivot once with each other key, the exact expectation is `2(n+1)H_n−4n`. Randomness comes from the algorithm.
- Three-way partition removes equal keys together. The distinct-key exact formula does not count comparisons in every three-way implementation.

Expected quicksort comparisons and bounded recursion depth

```
quicksort_small_side(A,l,r):
    while r-l>=2:
        p=uniformly_random_value_from(A[l:r])
        (lt,gt)=partition(A,l,r,p)
        if lt-l<r-gt:
            quicksort_small_side(A,l,lt); l=gt
        else:
            quicksort_small_side(A,gt,r); r=lt
```

Rank the distinct keys $1, \dots, n$, and let X_{ij} indicate whether ranks i, j are ever compared directly. The classic two-way version compares each pivot exactly once with every other key in its subproblem. Two keys are compared exactly when the first pivot chosen from the rank interval $[i, j]$ is one of its endpoints. The $j - i + 1$ keys are symmetric, so $\Pr(X_{ij} = 1) = 2/(j - i + 1)$. By linearity of expectation,

$$\mathbb{E}[C] = \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2(n+1)H_n - 4n = \Theta(n \log n),$$

where $H_n = \sum_{k=1}^{n} 1/k$.

Expected quicksort comparisons and bounded recursion depth (continued)

The three-way implementation tests <, >, and = sequentially, so its distinct-key partitions still use only a bounded number of comparisons per key. Expected O(n log n) remains valid, but the exact classic comparison count does not transfer unchanged. Each pivot must be compared with every nonpivot key at least once. The same random rank splits therefore also give the indicator-sum lower bound, establishing expected Θ(n log n) for distinct keys. All-equal input needs only one three-way partition and Θ(n) work.

Space has a separate deterministic proof: the actual recursive child above is the smaller nonequal region, of length at most (r−l−1)/2. Active frame sizes therefore halve at each nesting, giving worst-case O(log n) frames. The larger side uses the current frame's while loop. Even repeated extreme pivots still scan total length n+(n−1)+…=Θ(n^2); bounded stack space does not remove that time worst case.

Recursion space and depth protection

- Recursing on both sides can have $\Theta(n)$ worst-case stack depth. In-place partition does not imply constant total auxiliary space.
- Recurse only on the smaller side and process the larger side in a loop. Each actual recursive subproblem is at most half as large, limiting stack space to $O(\log n)$.
- This improves space but does not eliminate $\Theta(n^2)$ worst-case comparison work.
- Introspective sorting additionally switches to heapsort when recursion becomes too deep. It is an engineering extension.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge sort and stable merging

- For sorted halves, compare their first unconsumed values and output the smaller. Choose the left value on equality to preserve stability.
- The output is the sorted merge of consumed input, and its last value is no larger than either unconsumed remainder.
- Merging `[2_A,5]` and `[2_B,4]` gives `[2_A,2_B,4,5]`. Taking right first on equality reverses the equal-key records.
- Standard array merging uses a $\Theta(n)$ buffer. The recurrence `T(n)=2T(n/2)+Θ(n)` gives $\Theta(n \log n)$ worst-case time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete stable merge and merge-sort

```
merge(L,R):
    i=0; j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i].key<=R[j].key: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    append remaining L[i:]; append remaining R[j:]
    return out
merge_sort(A):
    if len(A)<2: return A
    m=len(A)//2
    return merge(merge_sort(A[:m]),merge_sort(A[m:]))
```

Merging L=[2_A,5], R=[2_B,4] first takes A on equality, with cursors (1,0) and output [A]. Taking B gives (1,1) and [A,B]. Taking 4 gives (1,2) and [A,B,4]. Appending 5 completes [A,B,4,5]. Choosing the smaller remaining head proves order, and consuming every input exactly once proves permutation preservation.

The slicing merge_sort is semantic pseudocode. The shared-buffer C++ implementation avoids retaining needless copies across levels.

Stable merging, step by step

Stable merge: which record is chosen next?

L = [2A, 5] R = [2B, 4]

Step	Compare	Take	Output
1	2A = 2B	2A	[2A]
2	5 > 2B	2B	[2A, 2B]
3	5 > 4	4	[2A, 2B, 4]
4	R empty	5	[2A, 2B, 4, 5]

Choose the left item on ties so 2A stays before the later 2B.

Multiway merge and a winner tree

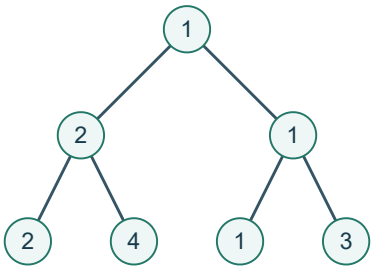
For k sorted streams totaling N records, read the first record of each nonempty stream. A winner tree stores current candidates at leaves and the smaller candidate at each internal node. After emitting the root, advance only its source and recompute the path from that leaf to the root. Treat exhaustion as a separate sentinel state larger than every valid key; the largest valid key is not infinity.

For [2,9], [4,8], [1,7], [3,6], the initial leaves [2,4,1,3] have winner 1. Emit 1 and replace it with 7: leaves [2,4,7,3] now have winner 2. Emit 2 and replace it with 9: the next winner is 3. Initialization costs O(k), each output O(log(k+1)), total O(k+N log(k+1)), and candidate storage O(k), excluding output and buffering.

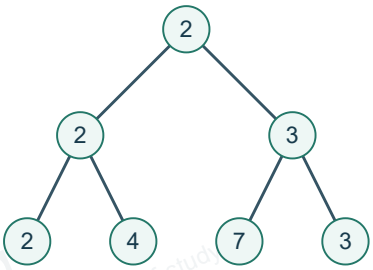
For a stable merge of runs that are contiguous in the original input, break equal-key ties by original run order and preserve each run's internal order. Another tie policy may produce sorted output while violating stability.

Updating a four-way winner tree

Winner tree: replay only the path from the changed leaf



Initial winner = 1



Emit 1, replace by 7: winner = 2

Each internal node keeps the smaller candidate; ties follow original run order.

Heapsort's sorted suffix

- For ascending order, build a max-heap, swap the root with the last heap element, and exclude that position from the heap.
- The prefix is a max-heap, the suffix is increasing, and every prefix key is at most every suffix key.
- The extracted root is the largest remaining key and belongs at the front of the sorted suffix. Sift down the replacement root.
- Construction is Θ(n). Extractions take Θ(n log n) worst-case total time. An iterative implementation uses O(1) extra space and is generally unstable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete heapsort driver and sorted-suffix trace

```
heap_sort(A):
    build_max_heap(A)
    for end=len(A)-1 down to 1:
        swap(A[0],A[end])
        sift_down_max(A,0,end)    // heap is [0,end), sorted suffix excluded
```

Heapsort on the same keys starts with max-heap [5,2,4,2]. Removing 5 gives [2,2,4|5], sifted to [4,2,2|5]. Removing 4 gives [2,2|4,5], then [2|2,4,5]. The fixed sorted suffix never reenters sifting. Equal value results do not establish record-identity stability.

Max-sifting chooses the larger child and stops when its parent is at least as large; indices match the min version with reversed comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting sort assumes a bounded key domain

- Integer keys in `[0,K)` use K counters. Prefix sums give the end position of each key's output block.
- Scan original records right to left, placing key k at `--end[k]`. This preserves equal-key order.
- Time and combined count/output space are Θ(n+K). A huge K relative to n may be impractical.
- Directly indexing by key obtains more than pairwise comparison information, so the pure comparison lower bound does not apply directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable counting allocation and record-by-record scatter

```
stable_count(records,K):
    out=array(len(records)); count=[0]*K
    for x in records: require 0<=x.key<K; count[x.key]+=1
    end=prefix_sum_inclusive(count)
    for x in reverse(records): end[x.key]-=1; out[end[x.key]]=x
    return out
```

For [(5,A),(2,B),(4,C),(2,D)],K=6, count=[0,0,2,0,1,1] and end=[0,0,2,2,3,4]. Right-to-left scattering sends D→out[1],C→out[2],B→out[0],A→out[3]. Output [B,D,C,A] preserves B before D. LSD passes require this stability, not an arbitrary swap-based equivalent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting sort: reverse scatter

Stable counting sort: scatter from right to left

Input: (5,A) (2,B) (4,C) (2,D)

count = [0,0,2,0,1,1] end = [0,0,2,2,3,4]

Scan	Destination	Output slots
D (2)	--end[2] = 1	[_, D, _, _]
C (4)	--end[4] = 2	[_, D, C, _]
B (2)	--end[2] = 0	[B, D, C, _]
A (5)	--end[5] = 3	[B, D, C, A]

B stays before equal-key D; end marks an exclusive right boundary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stability in LSD radix sort

- With base b and d digits, sort stably by each digit from least significant to most significant.
- After pass k, values are ordered by their lowest k digits. Stability in the next pass preserves lower-digit order within equal next digits.
- For `31,14,22,11`, the units pass gives `31,11,22,14`. A stable tens pass gives `11,14,22,31`.
- Stable counting sort costs $\Theta(n+b)$ per pass and $\Theta(d(n+b))$ overall. Signed values, variable-length strings, and Unicode need explicit encoding and ordering rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LSD driver and digit-order invariant

```
LSD(A,b,d):
    for position=0..d-1:
        A=stable_count_by_digit(A,position,b)
    return A
```

LSD assumes fixed width d and base b≥2, digit=(key//b^position) mod b, and nonnegative or order-preservingly encoded keys.

For 31,14,22,11, the units digits are 1,4,2,1, so the stable first pass gives 31,11,22,14. The tens digits are then 3,1,2,1, giving 11,14,22,31. Stability preserves lower-digit order inside equal next digits, proving the low-k-digit invariant. Arbitrary-precision division would invalidate constant-time digit extraction in Θ(d(n+b)).

Decision-tree lower bound

- For n distinct keys, assume a deterministic algorithm learns order only by comparing two keys. Comparison histories form a binary decision tree.
- All n! relative input permutations need distinguishable leaves. A height-h binary tree has at most 2^h leaves, so $h \geq \lceil \log_2(n!) \rceil$.
- Since $n! \geq (n/2)^{n/2}$, we have $\log_2(n!) = \Omega(n \log n)$. Worst-case comparisons are therefore Ω(n log n).
- The model concerns distinct keys and arbitrary permutations. It neither forbids linear sorting of bounded-domain keys nor requires n log n comparisons on every input.

Preserve qualifications in comparisons

Method	Best case	Worst case	Main auxiliary space	Stable
Insertion	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Yes, move only strictly greater keys
Randomized quicksort	Depends on partitions and duplicates	$\Theta(n^2)$, expected $\Theta(n \log n)$ for distinct keys	Stack can be $O(\log n)$	Generally no
Merge	$\Theta(n \log n)$, standard version here	$\Theta(n \log n)$	$\Theta(n)$	Yes, take left on equality
Heap	Depends on duplicates and implementation	$\Theta(n \log n)$	$O(1)$, iterative version	No
LSD radix	$\Theta(d(n+b))$, version here	Same	$\Theta(n+b)$	Every pass must be stable

Assume constant-time key operations. Incompatible cost models do not establish which method is faster.

Mistakes and verification

- Check order, the element multiset, and record stability where promised. is_sorted alone can miss data loss.
- Equal keys, sorted and reverse-sorted inputs, empty and singleton arrays, and many duplicates expose different partition boundaries.
- Use fixed-seed data and a reference sort to check negative and duplicate keys across input permutations.
- Executable cases are not a proof. Preserve partition invariants and recursive termination arguments.

References and attribution

Lin Cing, handwritten data structures notes; MIT 6.006 sorting and comparison lower bounds; ISO C++ sorting requirements.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study